

BACKEND CASE STUDY 137

The background of the slide is a dark, atmospheric night scene. It shows a wet, reflective street in the foreground. In the middle ground, there is a dark building with some lights on. A person is standing near the entrance of the building. To the right, there are blurred red and white light trails, suggesting motion, possibly from a car or a train. The overall mood is mysterious and modern.

ONLINE INTERVIEW SCHEDULING SYSTEM

Node.js, Express.js, MongoDB, Mongoose, JWT Authentication,
React Frontend

Presented by: Tanmay Vijay Sherkar

Scheduling breaks without backend rules

Recruitment teams need a reliable way to manage interview availability.

- Candidates booking already-reserved slots
- Candidates having overlapping interviews
- Recruiters publishing overlapping time slots
- Unauthorized users modifying interview slots
- Inconsistent validation between frontend and backend



Project objectives

01 Recruiters publish slots; candidates browse available ones

02 Prevent duplicate bookings and candidate and recruiter schedule conflicts

03 JWT-based authentication and role-based authorization

04 Validate all scheduling data on the backend

Two roles, two workflows



ROLE 01

Candidate

- Register and login
- View available interview slots
- Book an interview
- View personal interview schedule

ROLE 02

Recruiter

- Register and login
- Publish, edit, and cancel available slots
- View booked candidates

Three-layer architecture

LAYER 01

Frontend

React, Axios, candidate and recruiter dashboards



LAYER 02

Backend

Node.js, Express.js,
authentication and validation
middleware, scheduling logic



LAYER 03

Database

MongoDB Atlas,
Mongoose schemas,
User and
InterviewSlot
collections

Technology stack

LAYER	TECHNOLOGY
Frontend	React, Vite, Axios
Backend	Node.js, Express.js
Database	MongoDB Atlas
ODM	Mongoose
Authentication	JWT
Password security	bcrypt
Validation	express-validator
API testing	Postman / Thunder Client
Deployment	Render, Railway, Vercel, Netlify

Database design: two collections

COLLECTION

User

name, email, password, role, createdAt, updatedAt

COLLECTION

InterviewSlot

recruiter, candidate, startTime, endTime, status, title, meetingLink, createdAt, updatedAt

SLOT STATUS

AVAILABLE

BOOKED

CANCELLED

JWT authenticates, roles authorize

AUTHENTICATION

- Email and password registration, hashed with bcrypt
- Login returns a JWT, sent as a Bearer Authorization header

Authorization: Bearer <JWT_TOKEN>

AUTHORIZATION

- Recruiter-only slot creation
- Candidate-only slot booking
- Recruiter ownership check for editing and cancellation

REST API endpoints

METHOD	ENDPOINT	PURPOSE
POST	/api/auth/register	Register user
POST	/api/auth/login	Login user
GET	/api/auth/me	Get current user
GET	/api/slots	List available slots
GET	/api/slots/mine	View personal schedule
POST	/api/slots	Create recruiter slot
POST	/api/slots/:id/book	Book a slot
PATCH	/api/slots/:id	Update a slot
DELETE	/api/slots/:id	Cancel a slot

Overlapping intervals get rejected with 409

OVERLAP

**existingStartTime < requestedEndTime AND
existingEndTime > requestedStartTime**

- Recruiter conflict: before creating, find another active slot for the same recruiter and compare intervals
- Candidate conflict: before booking, compare the requested time with the candidate's existing bookings

ADJACENT SLOTS ARE ALLOWED

10:00–10:30

10:30–11:00

10:00–10:30 and 10:30–11:00 do not overlap